ASSIGNMENT PSEUDOCODE:

1. MAIN PROGRAM

```
START Program

    INITIALIZE GUI Window "Smart Healthcare Network Simulator"
    DISPLAY Header, Subtitle
    CREATE Buttons: [TOPOLOGY, IP/VLSM, ROUTING, PING, TCP/UDP,
                     HTTP/DNS, TRAFFIC, LINK_FAILURE, RUN_ALL]
    CREATE Output Console (text area)
    CREATE Status Bar

    ON Button Click:
        CALL corresponding module function
        DISPLAY result text in Output Console
        UPDATE Status Bar with completion message

    SHOW Topology by default on startup

    LOOP waiting for user interaction (event loop)

END Program
```

---

2. TOPOLOGY MODULE

```
FUNCTION show_topology():
    DEFINE network nodes:
        R1 = Core Router (Hospital Campus)
        SW1 = Core Switch connected to R1
        ICU, OPD, PHARMACY = Departments connected to SW1
        HIS_SERVER = Hospital Information System connected to SW1
        WAN = link between Hospital Campus and remote sites
        R2 = Router for Remote Clinic (Telemedicine)
        R3 = Router for Diagnostic Lab (Radiology)
        SW2 = Switch under R2, connects Remote Clinic devices + IoT monitors
        SW3 = Switch under R3, connects Diagnostic Lab devices + IoT sensors

    BUILD ASCII diagram connecting all nodes hierarchically
    PRINT diagram + design notes (topology type, device count, rationale)
    SET status = "TOPOLOGY COMPLETED"
END FUNCTION
```

---

3. IP ADDRESSING / VLSM MODULE

```
FUNCTION show_ip_vlsm():
    base_network = "192.168.20.0/24"
```

```
segments = [
    (name="OPD",            hosts_needed=60),
    (name="Pharmacy",       hosts_needed=30),
    (name="ICU",            hosts_needed=14),
    (name="HIS Server",     hosts_needed=14),
    (name="Remote Clinic",  hosts_needed=14),
    (name="Diagnostic Lab", hosts_needed=14),
    (name="WAN R1-R2",      hosts_needed=2),
    (name="WAN R1-R3",      hosts_needed=2)
]

SORT segments BY hosts_needed DESCENDING   // VLSM best practice

next_available_address = base_network.first_address
FOR each segment IN segments:
    required_bits = CEIL(LOG2(segment.hosts_needed + 2))  // +2 for network/broadcast
    subnet_size = 2 ^ required_bits
    subnet = ALLOCATE_BLOCK(next_available_address, subnet_size)
    segment.subnet = subnet
    segment.usable_range = subnet.first_host TO subnet.last_host
    next_available_address = subnet.next_free_address

VALIDATE no subnet overlaps
VALIDATE total_allocated_hosts <= 254

PRINT subnet allocation table
PRINT validation results (PASS/FAIL)
SET status = "IP / VLSM COMPLETED"
```

END FUNCTION

```

---

## 4. ROUTING MODULE

```
FUNCTION show_routing():
    routers = [R1, R2, R3]
    protocol = "OSPF Area 0"

    FOR each router IN routers:
        ADVERTISE directly connected subnets into OSPF
        FOR each neighbor_router:
            ESTABLISH OSPF adjacency (state = FULL)
            EXCHANGE Link-State Advertisements (LSAs)
            RUN Dijkstra Shortest Path First (SPF) algorithm
            BUILD routing table with best paths

    MEASURE convergence_time
    FOR each subnet IN all_subnets:
        VERIFY subnet is reachable from every router
        IF unreachable:
            MARK route as FAIL
        ELSE:
            MARK route as PASS
```
```

```
    PRINT routing tables per router
    PRINT OSPF neighbor adjacency states
    PRINT convergence_time
    SET status = "ROUTING COMPLETED"
END FUNCTION
```

---

5. PING (CONNECTIVITY) MODULE

```
FUNCTION show_ping():
    test_pairs = [
        (ICU_PC, HIS_Server),
        (OPD_PC, HIS_Server),
        (Pharmacy_PC, HIS_Server),
        (HIS_Server, Remote_Clinic),
        (HIS_Server, Diagnostic_Lab),
        (Remote_Clinic_IoT_Monitor, HIS_Server)
    ]

    total_packets_sent = 0
    total_packets_lost = 0

    FOR each (source, destination) IN test_pairs:
        FOR i = 1 TO 4:                    // simulate 4 ICMP echo requests
            total_packets_sent += 1
```

```
            latency = COMPUTE_SIMULATED_LATENCY(source, destination)
            IF route_exists(source, destination) AND link_is_up:
                result = "SUCCESS"
            ELSE:
                result = "REQUEST TIMED OUT"
                total_packets_lost += 1
        record average_latency, result FOR (source, destination)

    packet_loss_percent = (total_packets_lost / total_packets_sent) * 100
    PRINT per-pair ping results with avg latency
    PRINT overall packet_loss_percent
    SET status = "PING COMPLETED"
END FUNCTION
```

---

6. TCP / UDP MODULE

```
FUNCTION show_tcp_udp():

    // --- TCP simulation (used for EHR / patient record sync) ---
    FUNCTION simulate_tcp(source, destination, data):
        PERFORM 3-way handshake: SYN -> SYN-ACK -> ACK
        IF handshake_successful:
            connection_state = "ESTABLISHED"
            SEND data IN ordered segments
```

```
        FOR each segment:
            IF segment_lost:
                RETRANSMIT segment
        confirm ordered, reliable delivery
        RETURN status = "PASS"

    // --- UDP simulation (used for live ICU vitals / telemedicine streams) ---
    FUNCTION simulate_udp(source, destination, data):
        connection_state = "CONNECTIONLESS"          // no handshake
        SEND data AS discrete datagrams
        NO retransmission on loss (tolerate minor loss for real-time speed)
        RETURN status = "PASS"

    CALL simulate_tcp(OPD_PC, HIS_Server, "patient_record_update")
    CALL simulate_udp(ICU_Monitor, HIS_Server, "vitals_stream")

    PRINT TCP result (reliable, ordered, use case: EHR)
    PRINT UDP result (low overhead, use case: live vitals/streaming)
    SET status = "TCP / UDP COMPLETED"
END FUNCTION
```

---

7. HTTP / DNS MODULE

```
FUNCTION show_http_dns():
```

```
    dns_records = {
        "his.hospital.local"    : HIS_Server.ip,
        "clinic.hospital.local" : Remote_Clinic.ip,
        "lab.hospital.local"    : Diagnostic_Lab.ip
    }

    // DNS Resolution
    FOR each (hostname, ip) IN dns_records:
        resolved_ip = DNS_LOOKUP(hostname)
        IF resolved_ip == ip:
            PRINT hostname + " -> " + ip + " : RESOLVED"
        ELSE:
            PRINT hostname + " : RESOLUTION FAILED"

    // HTTP Request to Patient Portal
    request_url = "http://his.hospital.local"
    resolved_ip = DNS_LOOKUP("his.hospital.local")
    response = SEND_HTTP_GET(resolved_ip)
    IF response.status_code == 200:
        PRINT "HTTP SERVICE: Web response received"
    ELSE:
        PRINT "HTTP SERVICE: FAILED"

    SET status = "HTTP / DNS COMPLETED"
END FUNCTION
```

---

8. TRAFFIC / PERFORMANCE MODULE

```
FUNCTION show_traffic():
    conditions = ["Normal", "Peak OPD Hours", "Emergency Surge"]
    load_factor = {"Normal": 1, "Peak OPD Hours": 2.5, "Emergency Surge": 4}

    base_latency = 15         // ms
    base_throughput = 100     // Mbps
    base_loss = 0.5           // %

    FOR each condition IN conditions:
        factor = load_factor[condition]
        latency = base_latency * factor_curve(factor)
        throughput = base_throughput / factor_curve(factor)
        packet_loss = base_loss * factor

        RECORD (condition, latency, throughput, packet_loss)

    PRINT performance table (condition, latency, throughput, packet_loss)
    PRINT observation: "as traffic load increases -> latency & loss increase,
                        throughput decreases"
    PRINT recommendation: "apply QoS to prioritize ICU/IoT UDP traffic
                           during peak/congested periods"
    SET status = "TRAFFIC COMPLETED"
END FUNCTION
```

---

9. LINK FAILURE / RECOVERY MODULE

```
FUNCTION show_link_failure():
    critical_link = LINK(R1.G0/1, SW1_ICU)

    // Before failure
    result_before = PING(ICU_PC, HIS_Server)
    PRINT "BEFORE FAILURE: " + result_before          // SUCCESS

    // Simulate failure
    critical_link.status = DOWN
    PRINT "LINK DOWN: R1 G0/1 -------- X -------- SW1 (ICU)"

    result_during = PING(ICU_PC, HIS_Server)
    IF critical_link.status == DOWN:
        result_during = "REQUEST TIMED OUT"
        TRIGGER alert("ICU vitals monitoring stream INTERRUPTED")
    PRINT "DURING FAILURE: " + result_during

    // Recovery via redundant path / restoring link
    IF backup_path_exists:
        ACTIVATE backup_path
    critical_link.status = UP
    result_after = PING(ICU_PC, HIS_Server)
```

```
    PRINT "RECOVERY: " + result_after              // SUCCESS

    PRINT "RESULT: FAILURE AND RECOVERY VERIFIED"
    SET status = "FAILURE COMPLETED"
END FUNCTION
```

---

10. RUN ALL MODULE

```
FUNCTION run_all():
    results = {}

    results["IPv4/VLSM"]          = show_ip_vlsm()        // returns PASS/FAIL
    results["Routing"]             = show_routing()
    results["Ping"]               = show_ping()
    results["TCP"]                 = simulate_tcp(...)
    results["UDP"]                 = simulate_udp(...)
    results["HTTP"]                = show_http_dns() [http part]
    results["DNS"]                 = show_http_dns() [dns part]
    results["Traffic"]             = show_traffic()        // TESTED
    results["Link Failure/Recovery"]= show_link_failure()    // TESTED

    PRINT consolidated summary table of all results (PASS/FAIL/TESTED)
    PRINT traffic table (Normal / Peak / Congested)
    PRINT "FINAL STATUS: NETWORK TESTING COMPLETED"
```

```
    SET status = "ALL TESTS COMPLETED"
END FUNCTION
```

---

11. SUPPORTING ALGORITHMS (used across modules)

```
FUNCTION COMPUTE_SIMULATED_LATENCY(source, destination):
    IF source.department == destination.department:
        RETURN random(1, 5)          // same LAN segment, low latency
    ELSE IF path_includes_WAN(source, destination):
        RETURN random(15, 30)        // WAN hop, higher latency
    ELSE:
        RETURN random(2, 10)         // routed but within campus

FUNCTION ALLOCATE_BLOCK(start_address, size):
    // rounds start_address up to a boundary divisible by 'size'
    aligned_start = ROUND_UP(start_address, size)
    block = NEW Subnet(aligned_start, size)
    RETURN block

FUNCTION DNS_LOOKUP(hostname):
    IF hostname IN dns_table:
        RETURN dns_table[hostname]
    ELSE:
        RETURN NULL
```

```

---

## 12. OVERALL SYSTEM FLOW (End-to-End)

```
START
    BUILD Topology (departments, server, WAN, remote sites)
    ASSIGN IP addressing using VLSM
    CONFIGURE routing (OSPF) between routers
    VERIFY connectivity using Ping
    ANALYZE protocols: TCP (EHR/reliable) vs UDP (vitals/real-time)
    VERIFY application services: HTTP (portal) + DNS (name resolution)
    ASSESS performance under Normal / Peak / Congested traffic loads
    TEST fault tolerance via Link Failure + Recovery on critical ICU link
    AGGREGATE all results into final PASS/FAIL verification report
END
```
```